CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 9

Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

1. Aim and Objectives

To build a secure StudentHub user registration page in PHP that validates the input, refuses duplicate email addresses and usernames, stores the password only as a hash created with password_hash(), and inserts the new user into MySQL using MySQLi prepared statements.

- Validate every field in the browser (quick feedback) and again in PHP (the browser can be bypassed).
- Refuse an email address or username that is already registered – in PHP and with UNIQUE keys in MySQL.
- Store passwords only as password_hash() hashes; never show, log or save a plaintext password.
- Use MySQLi prepared statements for every query that contains user input.
- Intermediate extension: prepared statements. Advanced extension: CSRF token validation and an audit-log entry for every successful registration. Both are implemented.
- Demonstrate the seven evaluation requirements with genuine screenshots and automated tests.

2. Problem Definition

Official title: *"Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing."*

A registration form is the first part of a portal that an attacker can reach. Without server-side checks it accepts empty or invalid data; without duplicate checks one email address can own several accounts; plaintext passwords are exposed by any database leak; and SQL built by joining user input allows SQL injection. The form must therefore be easy to use for students and safe against these problems.

The practical must demonstrate seven evaluation requirements:

#	Requirement	Where it is shown
1	Successful user registration	Section 13, Figure 13
2	Empty-field validation	Section 7, Figure 6
3	Invalid email validation	Section 7, Figure 7
4	Password mismatch validation	Section 7, Figure 8
5	Duplicate email / username validation	Section 8, Figures 9 and 10
6	Password stored as a hash in MySQL	Section 9, Figure 11
7	Successful database insertion	Section 13, Figure 14

Starting point: the Practical-9 folder already contained a basic MySQLi registration (register.php, process_registration.php, users.php). It used password_hash() and prepared statements, but it stored users in the shared user_registration database that Practicals 10 and 14 also use, had no CSRF protection, no full-name or terms fields, a 6-character password minimum, one combined "username or email already exists" message, printed raw MySQL errors in the page, and users.php listed every user's email address without any login. The useful parts were kept (MySQLi, password_hash(), prepared statements) and the rest was rebuilt. The original files were backed up before the changes.

3. Tools and Technologies

Tool	Use in this practical
PHP 8.2.12 (XAMPP)	Server-side validation, sessions, CSRF, password_hash() / password_verify(); built-in web server for testing
MySQLi extension	Database access with prepared statements and transactions (required by the practical)
MariaDB 10.4.32 (XAMPP)	MySQL-compatible database server; database studenthub_prac9
phpMyAdmin 5.2.1	Importing schema.sql and showing the stored rows, hashes and audit log
HTML5, CSS3, JavaScript	Registration form, responsive layout, client-side validation and password strength indicator
Microsoft Edge	Running the form; headless mode used for the screenshots and the console check
Python 3	tests/test_app.py – the end-to-end test script (standard library only)
Git / GitHub	Version control

4. Database Design

The practical uses its own database, studenthub_prac9, so the main StudentHub database and the databases of other practicals are never changed. It has two InnoDB tables with the utf8mb4_unicode_ci collation; all columns are NOT NULL.

4.1 Tables

Table	Columns	Keys and constraints
users	user_id INT UNSIGNED AUTO_INCREMENT, full_name VARCHAR(100), username VARCHAR(30), email VARCHAR(254), password_hash VARCHAR(255), role ENUM('student','admin') DEFAULT 'student', created_at DATETIME	PRIMARY KEY user_id; UNIQUE username; UNIQUE email; CHECK username format; CHECK email format; CHECK that password_hash looks like a bcrypt or Argon2 hash
audit_logs	audit_id INT UNSIGNED AUTO_INCREMENT, user_id, action VARCHAR(50), created_at DATETIME	PRIMARY KEY audit_id; FOREIGN KEY user_id → users.user_id (ON DELETE RESTRICT, ON UPDATE CASCADE)

Design decisions: password_hash is VARCHAR(255) because PHP's documentation recommends room for longer future algorithms; the collation ignores letter case, so Demo_Asha is a duplicate of demo_asha; the CHECK on password_hash means that even a programming mistake cannot store a plaintext password; RESTRICT on the foreign key keeps the audit history from being deleted together with a user.

4.2 ER Diagram

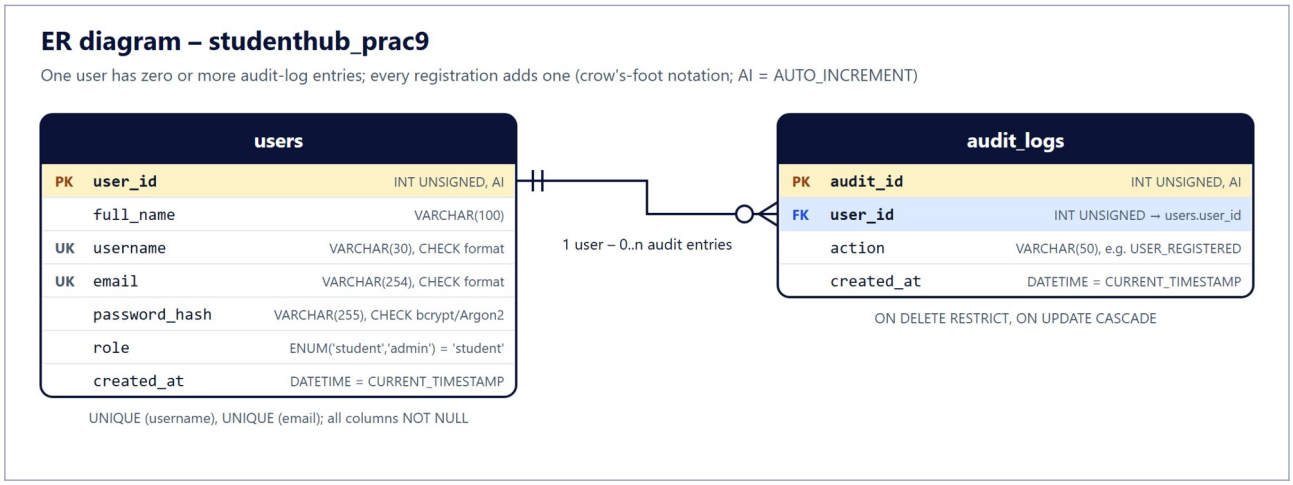


Figure 1: ER diagram of studenthub_prac9 (docs/er-diagram.png)

4.3 SQL (schema.sql)

```
CREATE TABLE IF NOT EXISTS users (
    user_id       INT UNSIGNED  NOT NULL AUTO_INCREMENT,
    full_name     VARCHAR(100)  NOT NULL,
    username      VARCHAR(30)   NOT NULL,
    email         VARCHAR(254)  NOT NULL,
    password_hash VARCHAR(255)  NOT NULL,            -- output of password_hash(), never the password itself
    role          ENUM('student', 'admin') NOT NULL DEFAULT 'student',
    created_at    DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id),
    UNIQUE KEY uq_users_username (username),         -- the collation ignores case: Asha_1 = asha_1
    UNIQUE KEY uq_users_email (email),
    CONSTRAINT chk_users_username CHECK (username REGEXP '^[A-Za-z][A-Za-z0-9_]{2,29}$'),
    CONSTRAINT chk_users_email CHECK (email LIKE '%_@_%._%'),
    -- the database itself refuses a value that does not look like a bcrypt or Argon2 hash
    CONSTRAINT chk_users_password_hash CHECK ((password_hash LIKE '$2y$%' OR password_hash LIKE '$argon2%')
                                              AND CHAR_LENGTH(password_hash) >= 60)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_unicode_ci;
```

```
CREATE TABLE IF NOT EXISTS audit_logs (
    audit_id   INT UNSIGNED NOT NULL AUTO_INCREMENT,
    user_id    INT UNSIGNED NOT NULL,
    action     VARCHAR(50)  NOT NULL,
    created_at DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (audit_id),
    KEY idx_audit_logs_user (user_id),
    -- RESTRICT: a user who has audit entries cannot be deleted by accident, so the history is kept
    CONSTRAINT fk_audit_logs_user FOREIGN KEY (user_id) REFERENCES users (user_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_unicode_ci;
```

4.4 Fictional Sample Data and Import

schema.sql adds three fictional accounts (demo_asha, demo_kabir, demo_meera, all @example.com) with one audit entry each; they are used to demonstrate the duplicate checks. Their hashes were made with password_hash() from random passwords that were thrown away, so no one knows a password for them. The file is safe to import again: tables are created only if missing (CREATE TABLE IF NOT EXISTS) and the sample rows are added only if their username and email are not present (INSERT … SELECT … WHERE NOT EXISTS), so users registered through the form are kept.

The file was imported in phpMyAdmin (Import tab → Choose File → schema.sql → Import) in Microsoft Edge. phpMyAdmin reported “Import has been successfully finished, 7 queries executed”. On a re-import the notes #1007 (database exists) and #1050 (table already exists) are expected.

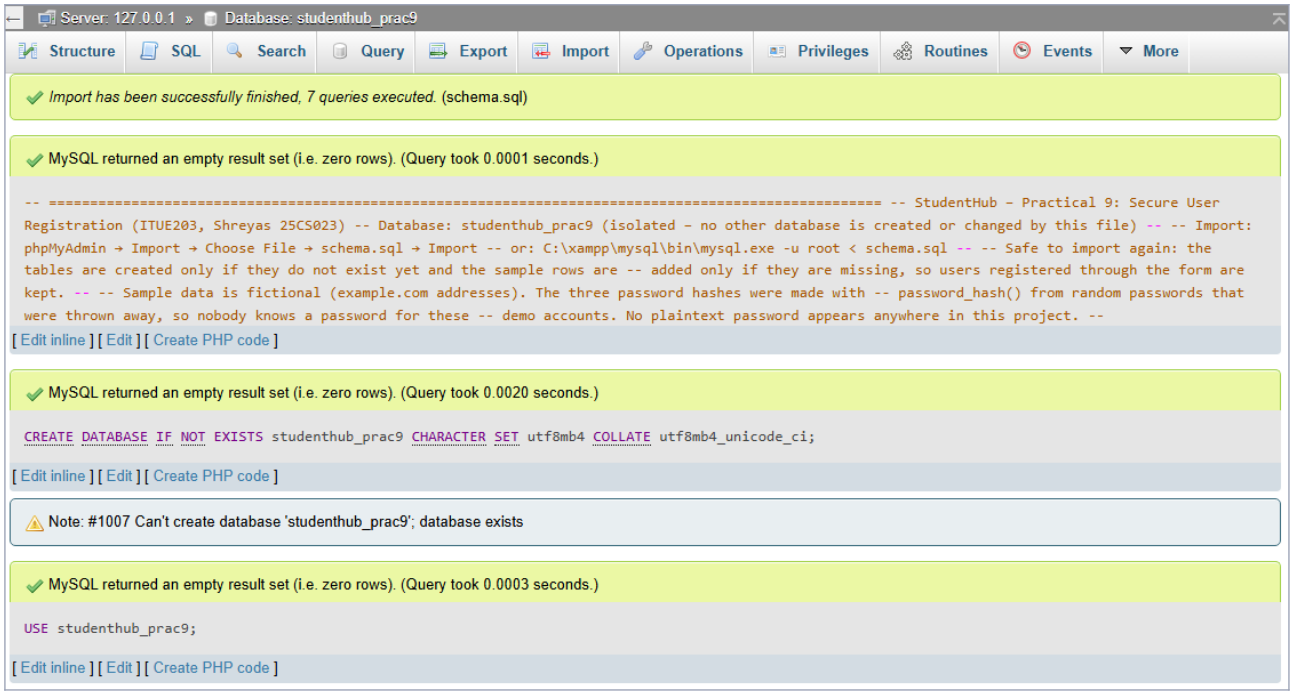


Figure 2: phpMyAdmin import of schema.sql into studenthub_prac9 (re-import; left panel cropped)

5. Registration Form Design

register.php shows one card with the six required inputs. Each input has a visible label, the right HTML5 type (email, password, checkbox), autocomplete hints and length limits. Hints and error messages are linked to the input with aria-describedby, invalid inputs get aria-invalid="true", and a hidden input carries the CSRF token. The layout is a single column of at most 620 px; a 480 px breakpoint adapts the spacing for phones (checked in Edge at 390 px width). The same navy header and footer as Practicals 5–8 keep the StudentHub look.

```
<form id="register-form" class="register-form" action="register.php" method="post">
    <input type="hidden" name="csrf_token" value="<?= e($token) ?>">

    <div class="field<?= isset($errors['full_name']) ? ' has-error' : '' ?>">
        <label for="full_name">Full name</label>
        <input type="text" id="full_name" name="full_name" value="<?= e($values['full_name']) ?>"
               required minlength="2" maxlength="100" autocomplete="name"<?= field_attrs('full_name',
$errors) ?>>
        <?= field_error('full_name', $errors) ?>
    </div>

    <div class="field<?= isset($errors['username']) ? ' has-error' : '' ?>">
        <label for="username">Username</label>
        <input type="text" id="username" name="username" value="<?= e($values['username']) ?>"
               required minlength="3" maxlength="30" pattern="[A-Za-z][A-Za-z0-9_]*"
autocomplete="username"
               autocapitalize="none" spellcheck="false"<?= field_attrs('username', $errors, true) ?>>
        <p class="field-hint" id="username-hint">3–30 characters: letters, numbers and _ , starting
with a letter.</p>
        <?= field_error('username', $errors) ?>
    </div>

    <div class="field<?= isset($errors['email']) ? ' has-error' : '' ?>">
        <label for="email">Email address</label>
        <input type="email" id="email" name="email" value="<?= e($values['email']) ?>"
               required maxlength="254" autocomplete="email" spellcheck="false"<?=
field_attrs('email', $errors) ?>>
        <?= field_error('email', $errors) ?>
    </div>
```

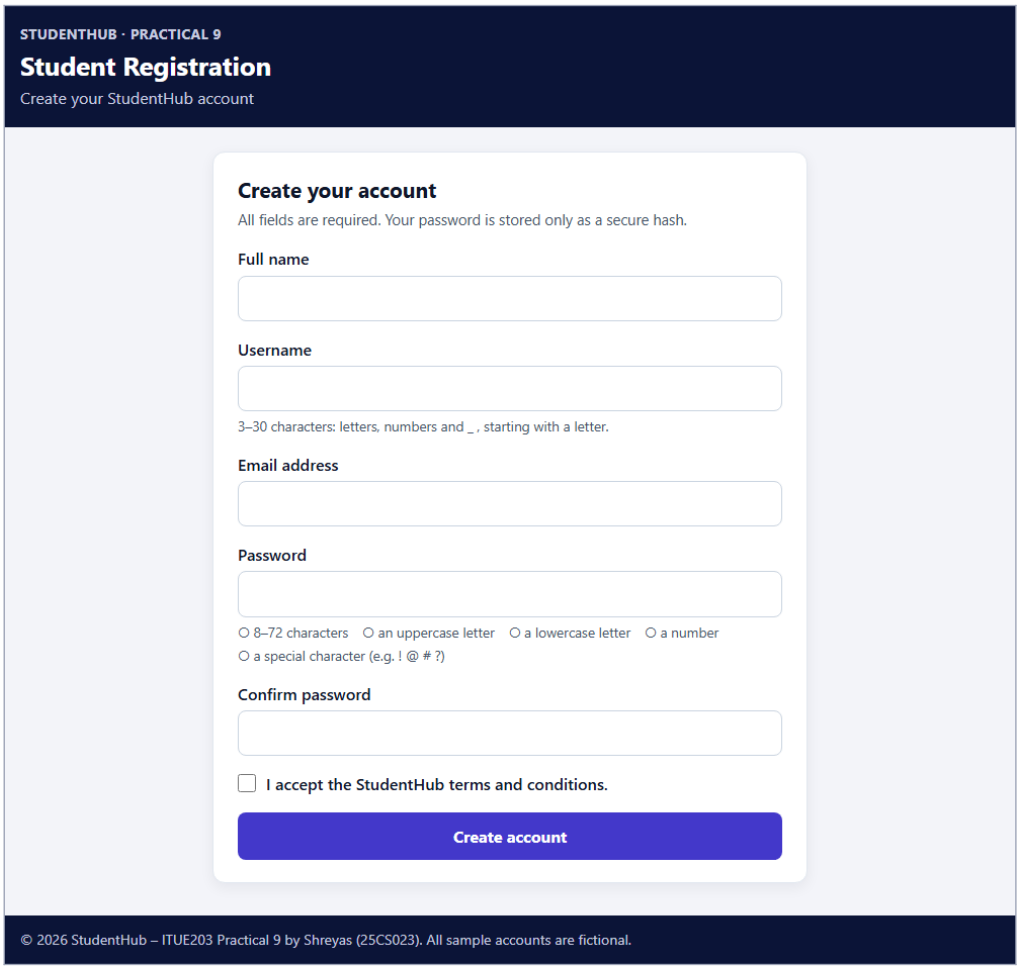


Figure 3: The registration form (screenshots/registration-form.png)

6. Frontend Validation

assets/js/register.js checks each field when the user leaves it, updates the message while the user corrects it, and checks the whole form on submit; if something is wrong it cancels the submission and moves the focus to the first invalid field. The messages are the same as the server's. The password strength indicator shows a coloured bar (Very weak, Weak, Fair, Good, Strong) and ticks each of the five password rules as soon as it is met. JavaScript only helps the user: if it is switched off or bypassed, PHP still performs every check (Section 7).

```
15  // Same rules as password_problems() in includes/registration.php
16  const passwordRules = {
17      length: (p) => p.length >= 8 && bytes(p) <= 72,
18      upper: (p) => /[A-Z]/.test(p),
19      lower: (p) => /[a-z]/.test(p),
20      number: (p) => /[0-9]/.test(p),
21      special: (p) => /[^A-Za-z0-9]/.test(p),
22  };
```

```
115  form.addEventListener('submit', (event) => {
116      const invalid = Object.keys(checks).filter((id) => !validate(id));
117      if (invalid.length) {
118          event.preventDefault();
119          field(invalid[0]).focus();
120      }
121  });
```

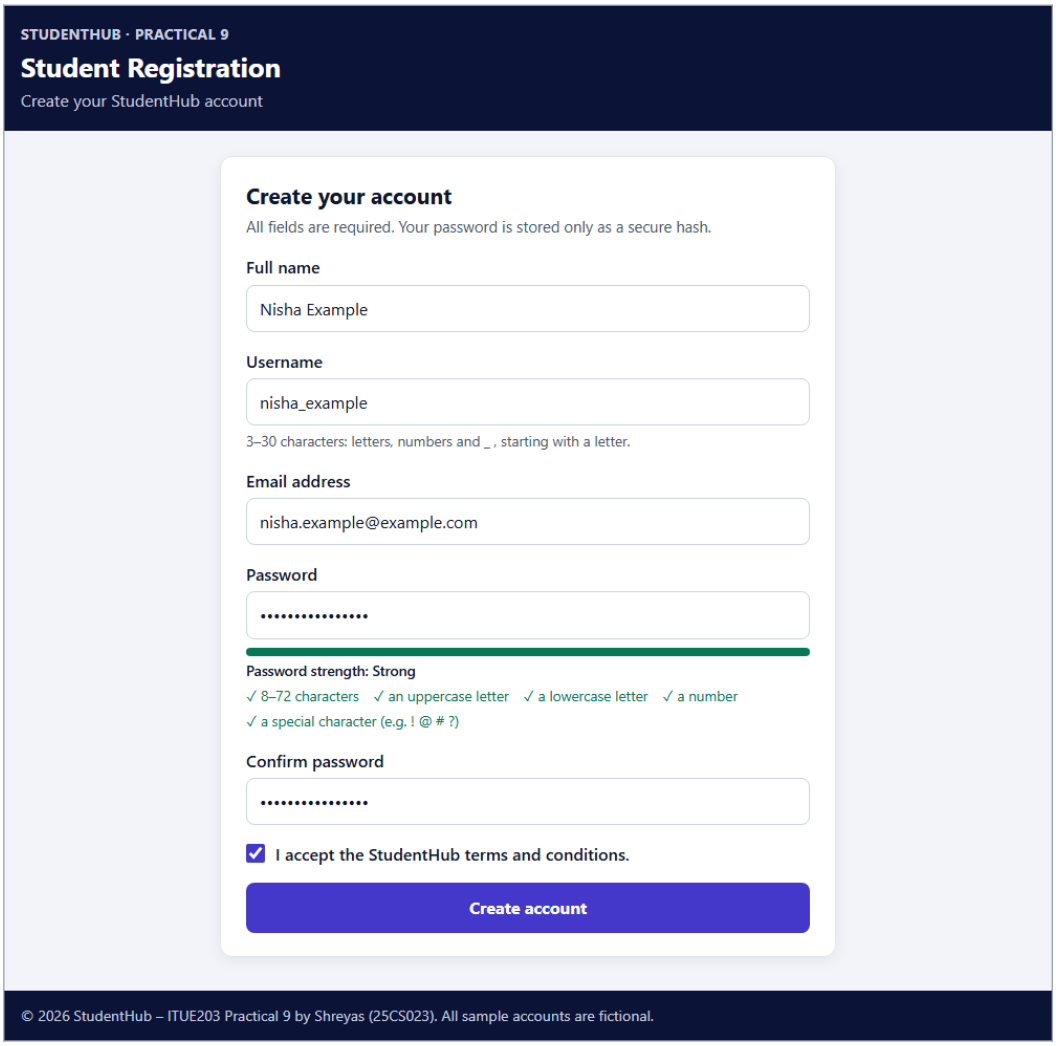


Figure 4: Password strength indicator – all five rules met (the password is masked)

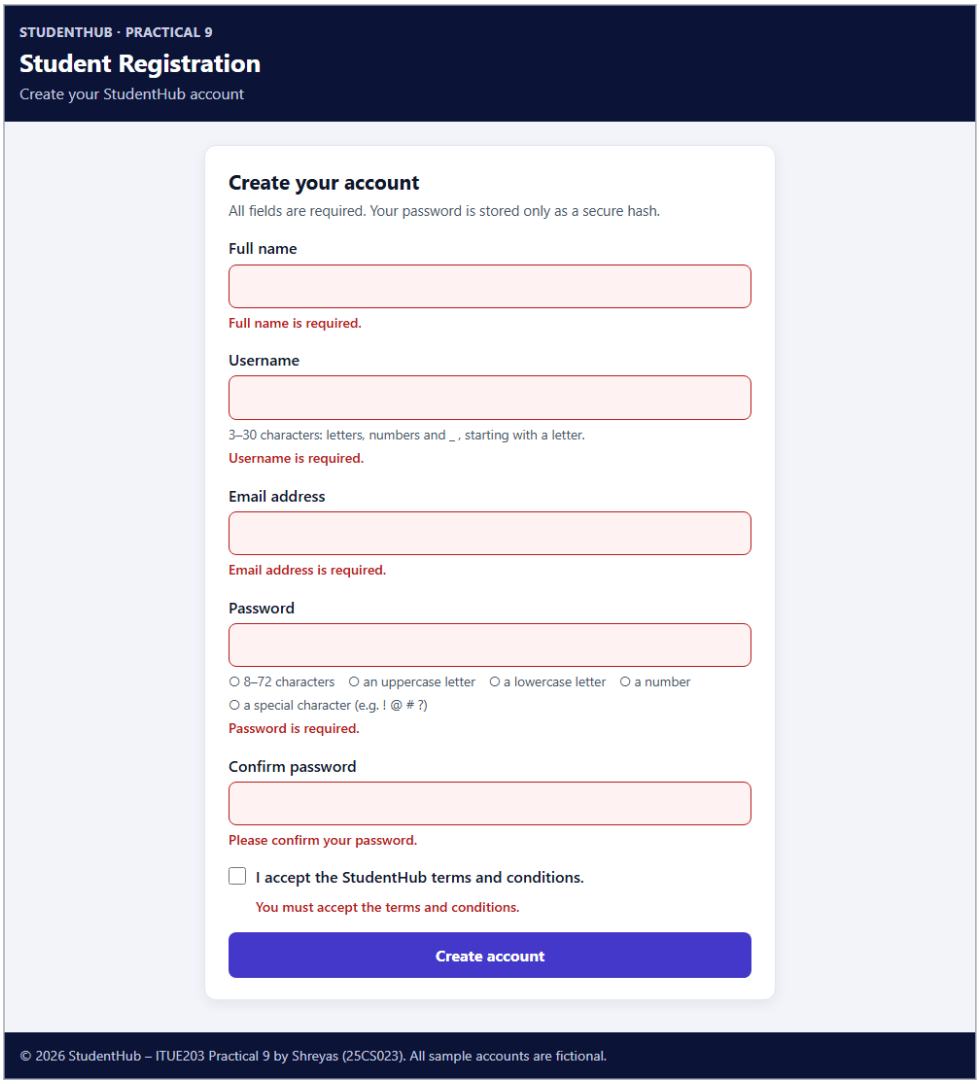


Figure 5: JavaScript (client-side) messages after pressing "Create account" on an empty form

7. Server-side Validation

validate_registration() in includes/registration.php checks every field independently of the browser. It first makes sure each value is a plain string (a request such as username[]=x sends an array) and valid UTF-8, trims the name, username and email (passwords are used exactly as typed), and then applies the rules: full name 2–100 letters, spaces, apostrophes, hyphens and full stops; username 3–30 characters starting with a letter; email checked with filter_var(FILTER_VALIDATE_EMAIL); password 8–72 characters with an uppercase letter, a lowercase letter, a number and a special character (bcrypt uses only the first 72 bytes); confirm password equal to the password; terms ticked. Any extra field, such as role=admin, is ignored.

```php
if (!isset($errors['email'])) {
    if ($email === '') {
        $errors['email'] = 'Email address is required.';
    } elseif (strlen($email) > 254 || filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
        $errors['email'] = 'Enter a valid email address, for example name@example.com.';
    }
}
if (!isset($errors['password'])) {
    if ($password === '') {
        $errors['password'] = 'Password is required.';
    } elseif ($problems = password_problems($password)) {
        $errors['password'] = 'Password must contain ' . implode(', ', $problems) . '.';
    }
}
if (!isset($errors['confirm_password'])) {
    if ($confirm === '') {
        $errors['confirm_password'] = 'Please confirm your password.';
    } elseif (!hash_equals($password, $confirm)) {
        $errors['confirm_password'] = 'Passwords do not match.';
    }
}
if (($input['terms'] ?? null) !== '1') {
    $errors['terms'] = 'You must accept the terms and conditions.';
}
```

When there are errors the page answers with HTTP 422, shows a summary at the top (with links to the fields) and the message under each field, and keeps the name, username, email and the terms tick. Passwords are never sent back. Every value is printed through htmlspecialchars(). For Figures 6–8 the form was sent with JavaScript's form.submit(), which skips the client-side check, so these screenshots show the PHP validation.

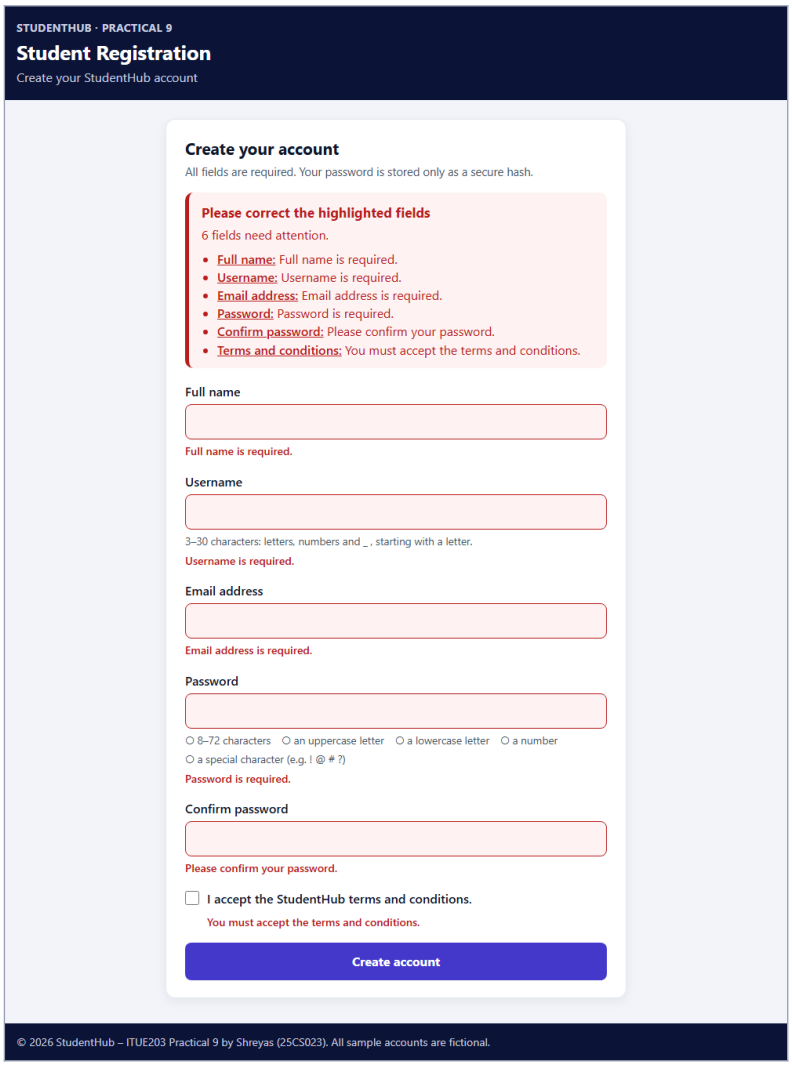


Figure 6: Requirement 2 – empty fields: server-side messages for all six fields (HTTP 422)

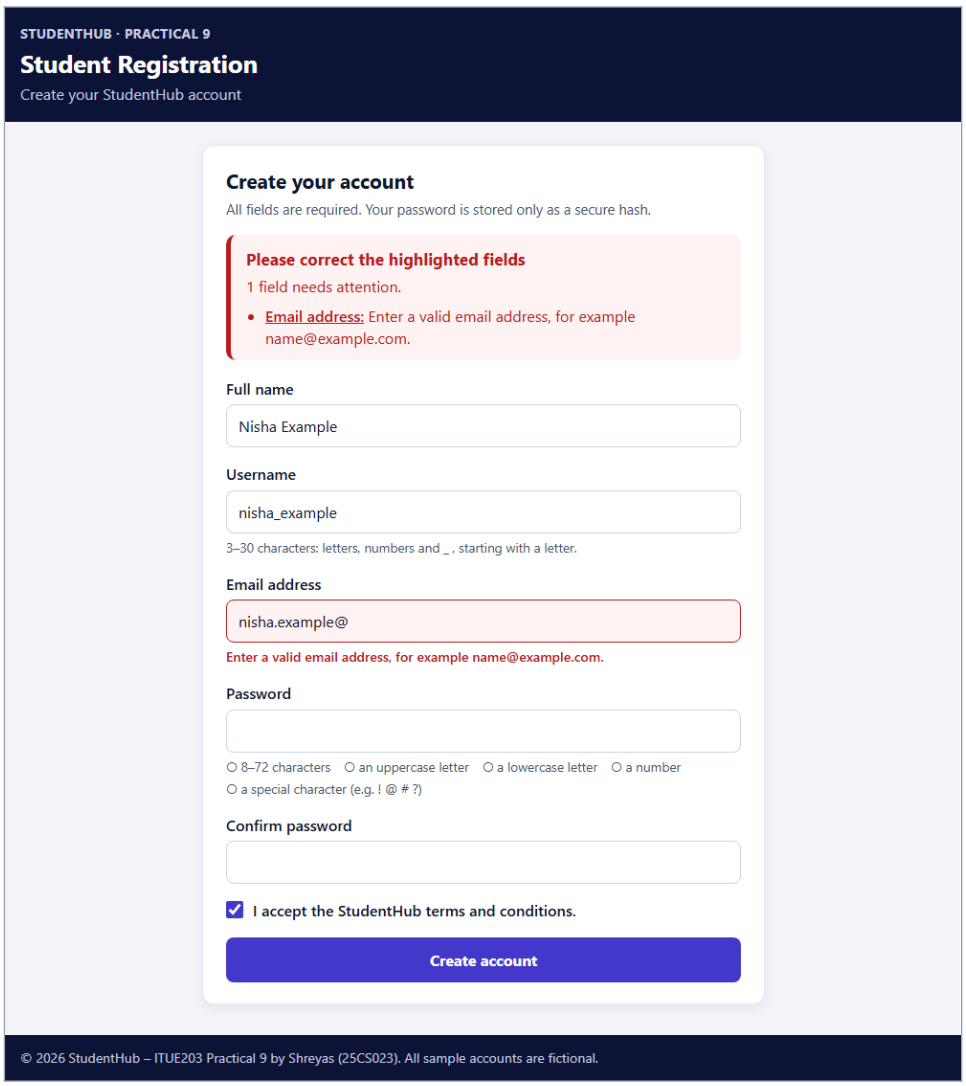


Figure 7: Requirement 3 – invalid email "nisha.example@" refused by the server; other values kept

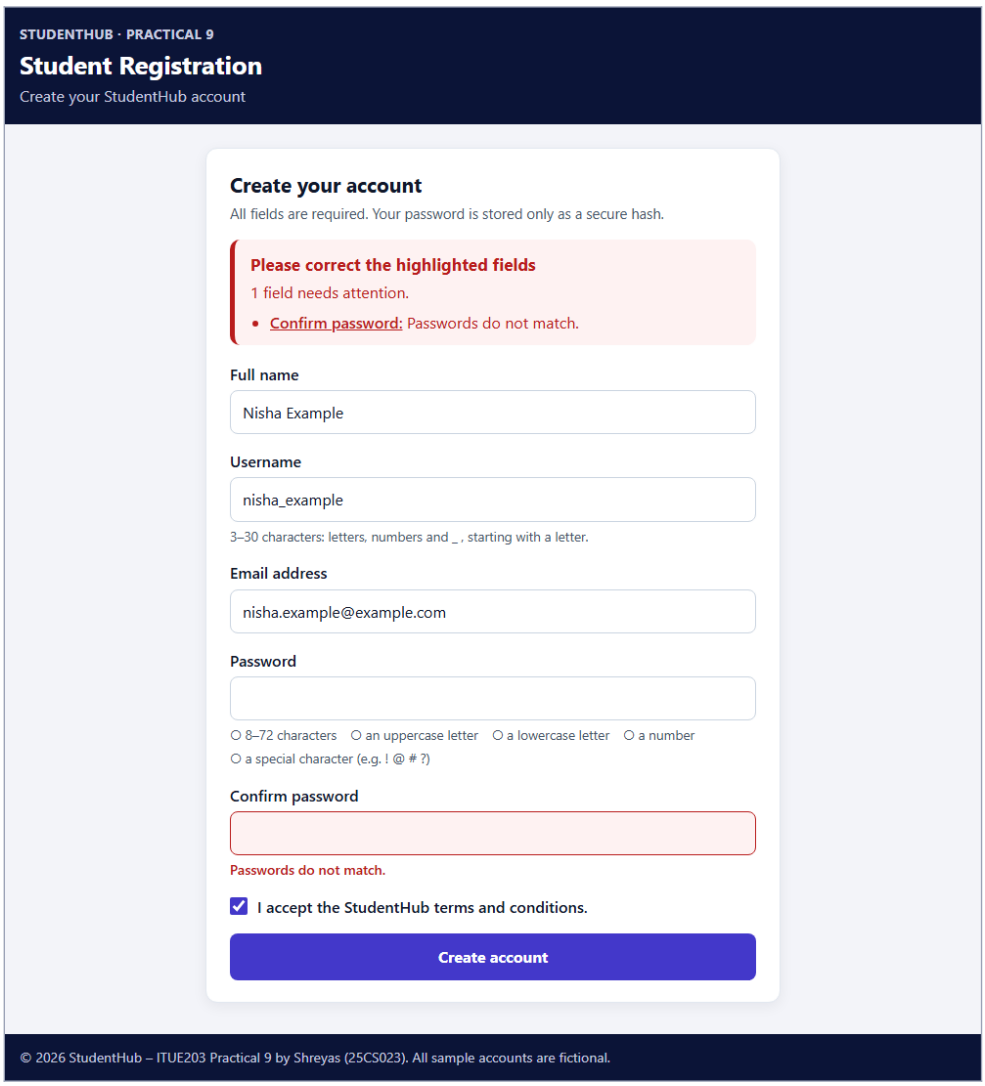


Figure 8: Requirement 4 – password mismatch refused by the server

8. Duplicate Email and Username Protection

After validation, find_duplicates() runs two separate prepared statements, one for the username and one for the email, so the user is told exactly which value is taken. Because the collation ignores case, DEMO_KABIR and Asha.Demo@Example.com are also found.

```
function find_duplicates(mysqli $db, string $username, string $email): array
{
    $errors = [];
    $stmt = $db->prepare('SELECT 1 FROM users WHERE username = ? LIMIT 1');
    $stmt->bind_param('s', $username);
    $stmt->execute();
    if ($stmt->get_result()->num_rows > 0) {
        $errors['username'] = 'This username is already taken. Please choose another one.';
    }
    $stmt->close();

    $stmt = $db->prepare('SELECT 1 FROM users WHERE email = ? LIMIT 1');
    $stmt->bind_param('s', $email);
    $stmt->execute();
    if ($stmt->get_result()->num_rows > 0) {
        $errors['email'] = 'An account with this email address already exists.';
    }
    $stmt->close();
    return $errors;
}
```

A check in PHP alone is not enough: two requests can both pass the check before either has inserted its row. The UNIQUE keys in MySQL therefore have the final word. If the INSERT fails with error 1062 (duplicate entry), create_user() rolls the transaction back and returns the same friendly message (code in Section 12). This race was reproduced in the tests with two database connections.

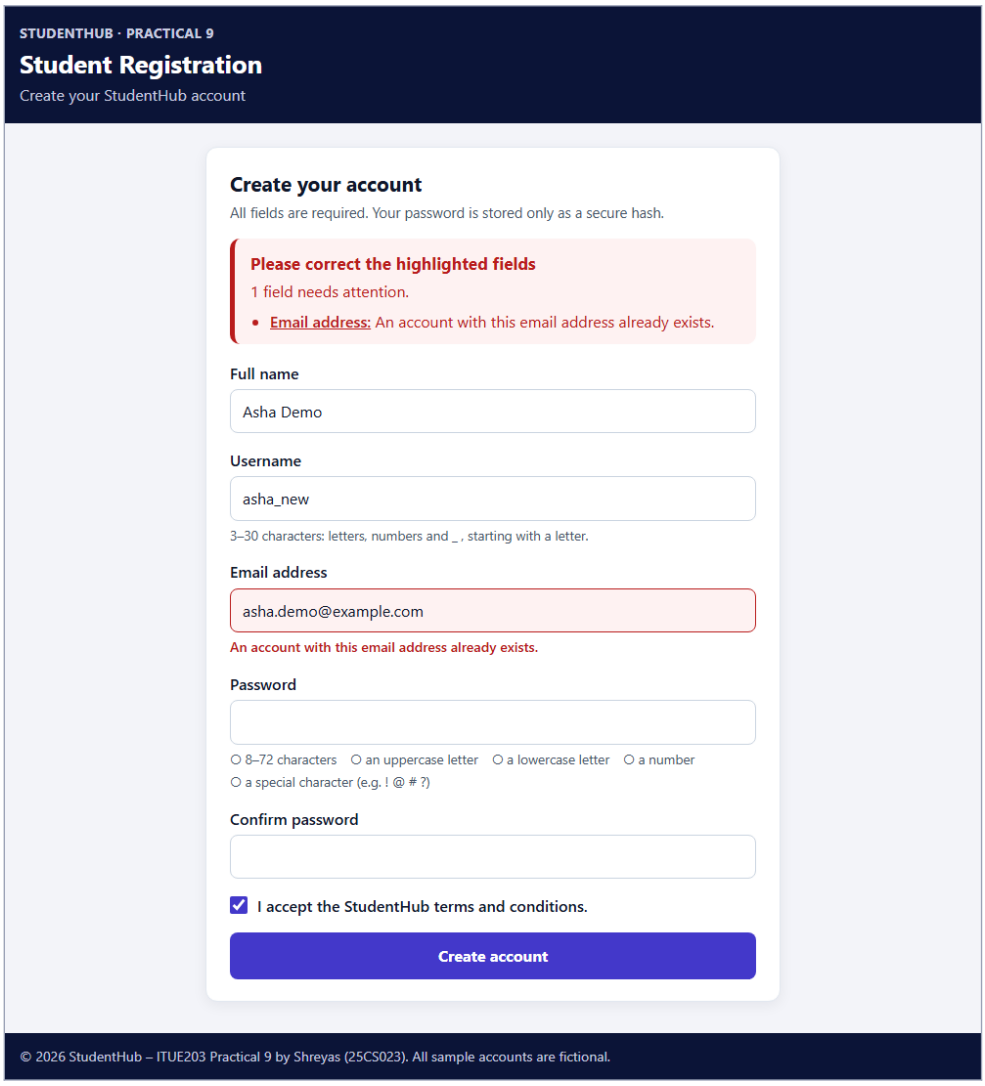


Figure 9: Requirement 5 – duplicate email asha.demo@example.com refused

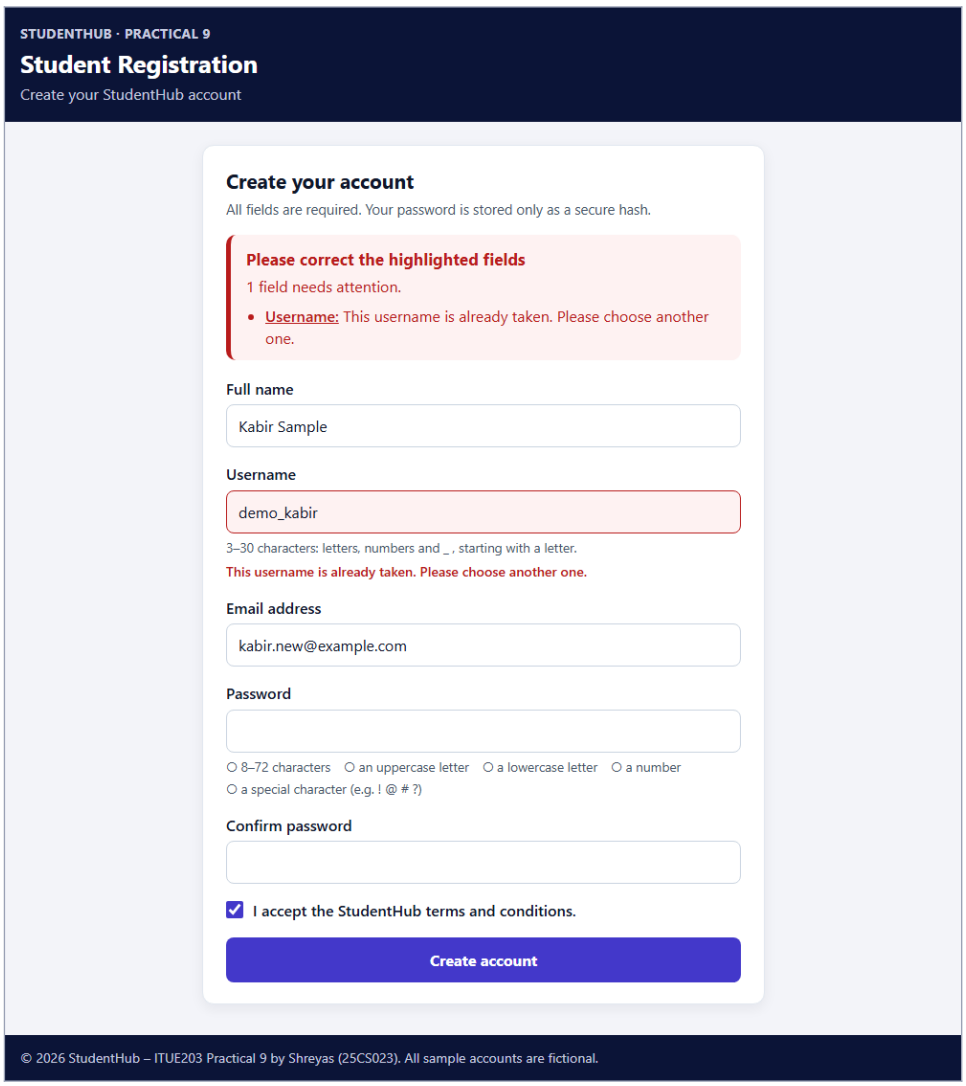


Figure 10: Requirement 5 – duplicate username demo_kabir refused

9. Password Hashing

A hash is a one-way function: the password can be turned into the hash, but the hash cannot be turned back into the password. password_hash($password, PASSWORD_DEFAULT) uses bcrypt in PHP 8.2 and returns a 60-character string such as $2y$10$… It contains the algorithm ($2y$), the cost (10, i.e. 2^10 rounds, which makes guessing slow) and a random salt, so the same password gives a different hash every time and precomputed tables are useless. To check a password at login, password_verify($password, $storedHash) recomputes the hash with the stored salt and compares the result.

```
169  function register_user(mysqli $db, array $clean): array
170  {
171      try {
172          $duplicates = find_duplicates($db, $clean['username'], $clean['email']);
173          if ($duplicates) {
174              return ['errors' => $duplicates];
175          }
176          $hash = password_hash($clean['password'], PASSWORD_DEFAULT);
177          return create_user($db, $clean['full_name'], $clean['username'], $clean['email'], $hash);
178      } catch (mysqli_sql_exception $exception) {
179          error_log('Practical 9 registration failed: [' . $exception->getCode() . '] ' . $exception-
     >getMessage());
180          throw new RuntimeException('Registration could not be completed.');
181      }
182  }
```

- Only the hash is stored; no password is shown in a page, written to a log, saved in a file or placed in this report or the screenshots.
- The CHECK constraint chk_users_password_hash makes MySQL refuse a value that does not look like a bcrypt/Argon2 hash (tested: error 4025).
- No reversible encryption is used – the server itself cannot recover a password.

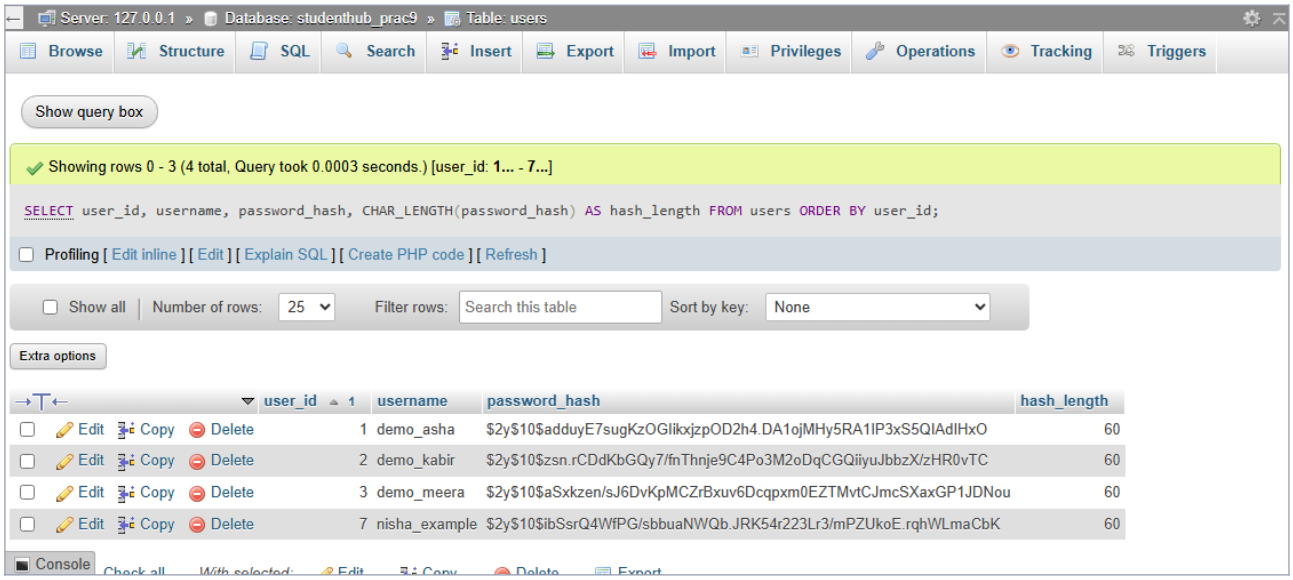


Figure 11: Requirement 6 – phpMyAdmin: password_hash holds 60-character bcrypt hashes, never the password (Full texts; left panel cropped)

Verification: in the tests, password_verify() returned true for the password used at registration and false for a different password, for every account registered by the tests (the passwords were random and never printed). Section 14.5 lists these checks.

10. MySQLi Prepared Statements

A prepared statement sends the SQL with ? placeholders first and the values separately, so a value can never change the meaning of the query. All queries in the application that contain user input use prepare(), bind_param() and execute(); nothing typed by the user is joined into SQL. db.php sets mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT), so every MySQL error throws an exception instead of being ignored, uses utf8mb4 and turns on strict SQL mode for the connection. A connection error is logged and replaced by a general RuntimeException.

```php
function get_db(): mysqli
{
    static $db = null;
    if ($db instanceof mysqli) {
        return $db;
    }
    mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);   // every MySQL error throws mysqli_sql_exception
    $config = db_config();
    try {
        $db = new mysqli((string) $config['host'], (string) $config['user'], (string) $config['pass'],
                         (string) $config['name'], (int) $config['port']);
        $db->set_charset('utf8mb4');
        // XAMPP's my.ini is not strict, so a too-long value would be cut off silently;
        // strict mode for this connection makes MySQL reject it with an error instead
        $db->query("SET SESSION sql_mode = 'STRICT_ALL_TABLES,ERROR_FOR_DIVISION_BY_ZERO,NO_ZERO_IN_DATE,NO_ZERO_DATE,NO_ENGINE_SUBSTITUTION'");
    } catch (mysqli_sql_exception $exception) {
        $db = null;
        error_log('Practical 9 database connection failed: [' . $exception->getCode() . '] ' . $exception->getMessage());
        throw new RuntimeException('The database connection could not be established.');
    }
    return $db;
}
```

Settings: XAMPP's defaults (root, empty password, studenthub_prac9) are used unless config.local.php (ignored by Git; template config.example.php) or P9_DB_* environment variables override them, so no real password is ever committed. db.php returns 403 when opened directly in the browser.

11. CSRF Protection and Session Security

Cross-site request forgery means another website making the visitor's browser submit this form. To prevent it, each session receives a random token of 32 bytes from random_bytes() (64 hexadecimal characters) which is placed in a hidden field. A POST is processed only if it sends exactly the token stored in the session; the comparison uses hash_equals(), which takes the same time whatever the input. A missing, wrong, array or other-session token gives HTTP 403 and nothing is validated or saved. After a successful registration the session ID is regenerated with session_regenerate_id(true) and a new token is created, so an old copy of the form cannot be sent again.

```php
// One random token per session (32 bytes from random_bytes, written as 64 hex characters)
function csrf_token(): string
{
    if (!isset($_SESSION['csrf_token']) || !is_string($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

// A request is accepted only if it sends exactly the token stored in this session.
// hash_equals() compares in constant time, so the comparison does not reveal how many characters matched.
function csrf_is_valid(mixed $submitted): bool
{
    $expected = $_SESSION['csrf_token'] ?? null;
    return is_string($expected) && is_string($submitted) && $submitted !== ''
        && hash_equals($expected, $submitted);
}

// New token after a successful registration, so an old copy of the form cannot be submitted again
function csrf_rotate(): void
{
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
```

The session cookie P9SESSID is HttpOnly (not readable by JavaScript), SameSite=Strict (not sent with requests from other sites) and Secure when HTTPS is used; session.use_strict_mode refuses session IDs not created by the server. Every page also sends a Content-Security-Policy (scripts and styles only from this site, no framing), X-Frame-Options: DENY, X-Content-Type-Options: nosniff and Cache-Control: no-store.

The controller in register.php shows the order of the checks and the HTTP status of each outcome:

```
21  if ($method === 'POST') {
22      if (!csrf_is_valid($_POST['csrf_token'] ?? null)) {
23          // Missing or wrong token: nothing is validated or saved, and the submitted values are not
    shown again
24          http_response_code(403);
25          $notice = ['title' => 'The form could not be submitted',
26                     'text' => 'Your session has expired or the request did not come from this form.
    Please fill in the form again.'];
27      } else {
28          [$clean, $errors] = validate_registration($_POST);
29          $values = ['full_name' => $clean['full_name'], 'username' => $clean['username'],
30                     'email' => $clean['email'], 'terms' => ($_POST['terms'] ?? null) === '1'];
31          if (!$errors) {
32              try {
33                  $result = register_user(get_db(), $clean);
34                  if (isset($result['user_id'])) {
35                      session_regenerate_id(true);          // new session ID after the account is
    created
36                      csrf_rotate();
37                      $_SESSION['registered'] = ['user_id' => $result['user_id'], 'full_name' =>
    $clean['full_name'],
38                                                 'username' => $clean['username'], 'email' =>
    $clean['email']];
39                      header('Location: register.php?registered=1', true, 303);   // Post/Redirect/Get:
    no resubmission on refresh
40                      exit;
41                  }
42                  $errors = $result['errors'];
43              } catch (RuntimeException $exception) {      // already logged in db.php or register_user()
44                  http_response_code(503);
45                  $notice = ['title' => 'Registration is not available right now',
46                             'text' => 'Your details were not saved because of a server problem. Please
    try again in a few minutes.'];
47              }
48          }
49          if ($errors) {
50              http_response_code(422);
51              $notice = ['title' => 'Please correct the highlighted fields',
52                         'text' => count($errors) === 1 ? '1 field needs attention.' : count($errors) . '
    fields need attention.'];
53          }
```

12. Audit Logging and Transaction

create_user() inserts the user and the audit entry inside one transaction. If either INSERT fails, rollback() removes both, so a user never exists without its audit entry and an audit entry never points to a missing user. The audit row stores only the user ID, the action USER_REGISTERED and the time – never a password or hash. Failed registrations (validation errors, duplicates, CSRF) never reach this function, so they create no entry.

```php
function create_user(mysqli $db, string $fullName, string $username, string $email, string $passwordHash): array
{
    $db->begin_transaction();
    try {
        $stmt = $db->prepare('INSERT INTO users (full_name, username, email, password_hash) VALUES (?, ?, ?, ?)');
        $stmt->bind_param('ssss', $fullName, $username, $email, $passwordHash);
        $stmt->execute();
        $userId = (int) $db->insert_id;
        $stmt->close();

        $action = AUDIT_REGISTERED;
        $stmt = $db->prepare('INSERT INTO audit_logs (user_id, action) VALUES (?, ?)');
        $stmt->bind_param('is', $userId, $action);
        $stmt->execute();
        $stmt->close();

        $db->commit();
        return ['user_id' => $userId];
    } catch (mysqli_sql_exception $exception) {
        $db->rollback();
        if ($exception->getCode() === 1062) {         // duplicate key: another request registered it first
            $key = str_contains($exception->getMessage(), 'uq_users_email') ? 'email' : 'username';
            return ['errors' => [$key => $key === 'email'
                ? 'An account with this email address already exists.'
                : 'This username is already taken. Please choose another one.']];
        }
        throw $exception;
    }
}
```

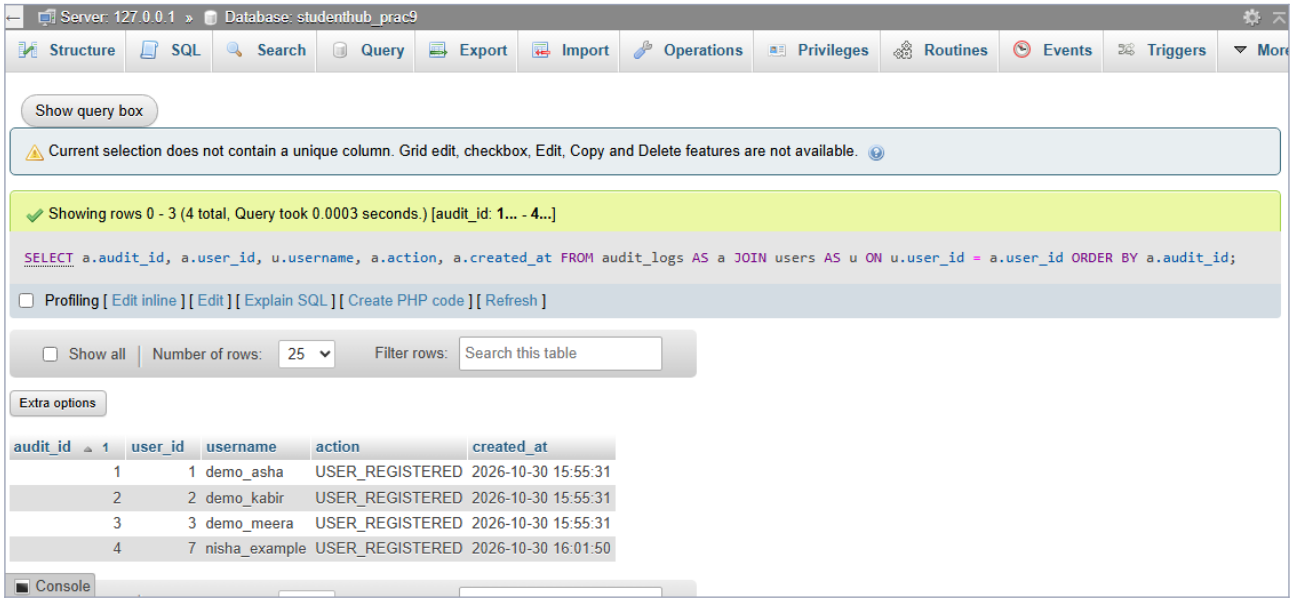


Figure 12: phpMyAdmin – audit_logs joined with users: one USER_REGISTERED entry per account, including nisha_example (left panel cropped)

The rollback was tested by renaming audit_logs in the test database so that the second INSERT failed: register_user() reported a general error, the real MySQL error 1146 was written to the error log, and the users row had been removed by the rollback.

13. Successful Registration and Database Insertion

The fictional account Nisha Example (nisha_example, nisha.example@example.com) was registered by typing into the form in Edge and pressing “Create account”. The JavaScript check passed, PHP validated the data, found no duplicate, hashed the password and committed the transaction, then redirected (HTTP 303)

to the confirmation page. Refreshing that page shows an empty form instead of submitting again (Post/Redirect/Get).

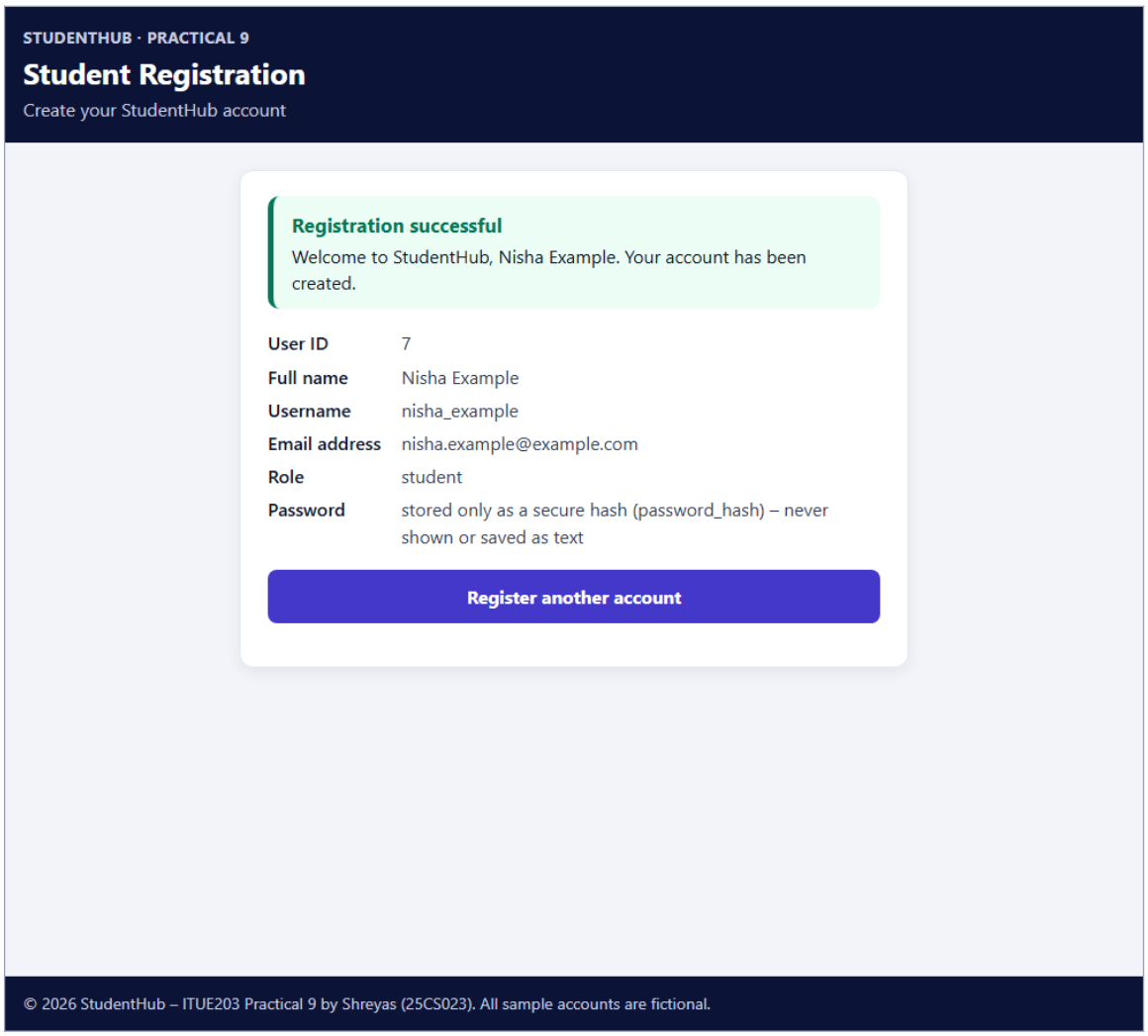


Figure 13: Requirement 1 – successful registration and confirmation

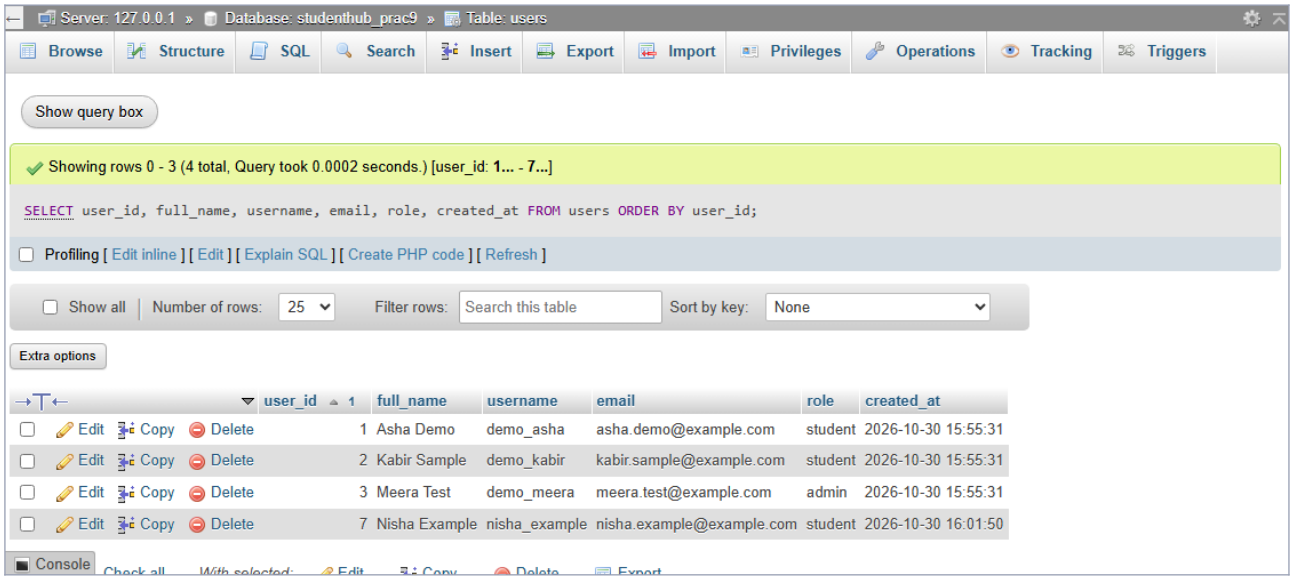


Figure 14: Requirement 7 – phpMyAdmin: the new row (user_id 7) in the users table, role student (left panel cropped)

The new account received user_id 7 instead of 4. An earlier version of schema.sql inserted the sample rows with INSERT … ON DUPLICATE KEY UPDATE, and each re-import used up AUTO_INCREMENT numbers even though no row was added. This was corrected (Section 15); the existing data was kept rather than deleting and recreating the database.

14. Testing

14.1 Method

All tests were executed on the final version with PHP 8.2.12 and MariaDB 10.4.32 on Windows 11. They use a separate database, studenthub_prac9_test, created from schema.sql (only the name is changed) and removed afterwards; a marker table ensures that the tests only ever drop a database they created themselves. studenthub_prac9 is therefore never changed by the tests. Test passwords are random and are never printed or saved.

Test suite	Checks	Result
tests/test_registration.php (PHP CLI: validation, duplicates, hashing, constraints, race condition, rollback)	93	93 passed, 0 failed
tests/test_app.py (Python: php -l on all PHP files, the PHP suite, HTTP tests on two PHP servers)	78	78 passed, 0 failed

In addition, the screenshots were taken from the real application with the real database, the browser console was checked in Edge, and schema.sql was imported with phpMyAdmin and with the MySQL command-line client (--show-warnings: only the expected notes 1007 and 1050 on a re-import).

14.2 Valid Registration Test Cases

#	Input	Expected	Actual
V1	Nisha Example / nisha_example / nisha.example@example.com, strong password, terms ticked (browser)	303, confirmation, row and audit entry	Pass (Figures 12–14)
V2	Zoë O'Neil / zoe_oneil (apostrophe and accented letter)	Stored exactly as typed	Pass
V3	Name with extra spaces “ Zoë O'Neil-Fernandes ”	Trimmed to “Zoë O'Neil-Fernandes”	Pass
V4	Password containing ' OR '1'='1	Registered; password_verify() true	Pass
V5	Five registrations in a row in one session	5 users, 5 hashes, 5 audit entries	Pass
V6	Extra field role=admin	Ignored, role is student	Pass

14.3 Invalid Registration Test Cases

#	Input	Expected	Actual
I1	All fields empty	422, six “required” messages	Pass (Figure 6)
I2	Name or username of spaces only	Treated as empty	Pass

#	Input	Expected	Actual
I3	Emails: plainaddress, name@, @example.com, name@example, na me@example.com, name@@example.com, not-an-email	“Enter a valid email address…”	Pass (Figure 7)
I4	Confirm password different	“Passwords do not match.”	Pass (Figure 8)
I5	Weak passwords: too short, no uppercase / lowercase / number / special, over 72 bytes	Password error listing the missing rules	Pass
I6	Usernames: ab, 31 characters, 1student, stu dent, stu-dent, ünï	Username format error	Pass
I7	Names: A, Name123, <script>alert(1)</script>	Name error; HTML shown escaped	Pass
I8	Terms missing, '', '0', 'on' or an array	“You must accept the terms…”	Pass
I9	username[]=x, invalid UTF-8	“must be sent as text” / “not allowed”; no PHP warning	Pass
I10	Missing / wrong / other-session / array CSRF token, or no session	403, nothing saved	Pass
I11	PUT request	405, Allow: GET, POST	Pass
I12	MySQL unreachable (wrong port)	503, general message, values kept, real error only in the log	Pass

14.4 Duplicate Registration Test Cases

#	Input	Expected	Actual
D1	Email asha.demo@example.com (and ASHA.Demo@Example.com)	“An account with this email address already exists.”	Pass (Figure 9)
D2	Username demo_kabir (and DEMO_KABIR)	“This username is already taken…”	Pass (Figure 10)
D3	Username and email both taken (demo_meera)	Both messages	Pass
D4	Registering multi_user_3 a second time	Refused (422)	Pass
D5	Race: B checks while A's insert is uncommitted, then A commits	B stopped by UNIQUE key (1062), friendly message, rolled back	Pass
D6	Direct SQL INSERT of a duplicate username / email / Demo_Asha	MySQL error 1062	Pass
D7	After all refused attempts	No new user, no new audit entry	Pass

14.5 Database Insertion and Password-hash Verification

Check (executed)	Result
users row contains the submitted full name, username, email and role student	Pass

Check (executed)	Result
password_hash starts with $2y$10$, is 60 characters long and is not the password	Pass
password_get_info() reports bcrypt	Pass
password_verify(registration password, stored hash)	true
password_verify(different password, stored hash)	false
Hashing the same password again gives a different hash (random salt)	Pass
No column of the new row contains the password	Pass
Exactly one USER_REGISTERED audit entry for the new user; audit_logs has no password/hash column	Pass
CHECK refuses plaintext in password_hash (4025); FOREIGN KEY refuses unknown user (1452) and keeps users with audit entries (1451); strict mode refuses a 101-character name (1406)	Pass
Error logs of all test servers contain no password and no $2y$ hash; no PHP warnings or notices	Pass

14.6 Results of the Seven Evaluation Requirements

#	Requirement	Actual result
1	Successful user registration	Pass – 303 and confirmation page (Figure 13); 5 further registrations in the tests
2	Empty-field validation	Pass – six messages, HTTP 422 (Figures 5, 6)
3	Invalid email validation	Pass – 8 invalid addresses refused (Figure 7)
4	Password mismatch validation	Pass (Figure 8)
5	Duplicate email / username validation	Pass – both, also in other letter case and in a race (Figures 9, 10)
6	Password stored in hashed format	Pass – bcrypt hashes, password_verify() true (Figure 11)
7	Successful database insertion	Pass – row in users and audit entry (Figures 14, 12)

Browser console: on all form pages in Edge there were no JavaScript errors, exceptions or Content-Security-Policy violations. The only entries were the expected “status of 422” network messages of the validation attempts. A missing favicon (404) found by this check was added.

Protected files: db.php, includes/, tests/, config.example.php, config.local.php, router.php, start-server.bat, .gitignore and .htaccess all return 403.

15. Problems Encountered and Fixes

- **Shared database:** the original code used user_registration, which Practicals 10 and 14 also use. The practical now uses its own studenthub_prac9 and the tests use studenthub_prac9_test.

- **Errors shown to users:** the original code printed connect_error and $stmt->error in the page. Errors are now logged and the user sees a general message.
- **Public list of users:** users.php showed every user's email address to anyone. It was removed; the database is inspected in phpMyAdmin instead.
- **Unclear duplicate message:** one query for “username OR email” could not say which was taken; there are now two checks with separate messages, plus handling of error 1062 for simultaneous requests.
- **AUTO_INCREMENT gap:** INSERT … ON DUPLICATE KEY UPDATE used up ID numbers on each re-import (user 7 instead of 4). The sample rows now use INSERT … SELECT … WHERE NOT EXISTS; three imports in a row were tested to leave AUTO_INCREMENT unchanged.
- **phpMyAdmin messages:** a server-level import shows “#1046 No database selected” from phpMyAdmin's own queries (as in Practical 8); importing from the database's Import tab avoids them. phpMyAdmin also shortens text to 50 characters, so the Full texts option was used to show the complete 60-character hashes.
- **Missing favicon:** found by the browser console check (404) and added.
- **Test-script problems (not application bugs):** the race-condition test first inserted a user without its audit row, and the HTTP test client did not reload the form after the one-time confirmation page; both test scripts were corrected.

16. Limitations

- Registration only: there is no login page, email verification or password reset.
- No rate limiting or CAPTCHA, so automated mass registration is not prevented.
- The duplicate-email message reveals that an address is registered (required by the practical; a production site might send an email instead).
- Tested with XAMPP's PHP built-in server, MariaDB 10.4 and Microsoft Edge only; XAMPP Apache (.htaccess), MySQL 8 and other browsers were not tested.

17. Results and Conclusion

All seven evaluation requirements were met and demonstrated with genuine screenshots and automated tests (93 + 78 checks, all passed). The registration form validates every field in the browser and again in PHP, refuses duplicate email addresses and usernames both in PHP and through UNIQUE keys in MySQL, and stores passwords only as bcrypt hashes created with password_hash(), which password_verify() confirmed. Every query with user input is a MySQLi prepared statement; injection-style input was refused or stored only as data. Both extensions were implemented: CSRF tokens from random_bytes() checked with hash_equals(), and an audit-log entry written in the same transaction as the user. The practical showed that security comes from several layers working together – browser checks for usability, server checks for correctness, and database constraints as the final guarantee.

18. References

- PHP Manual – password_hash: https://www.php.net/manual/en/function.password-hash.php
- PHP Manual – password_verify: https://www.php.net/manual/en/function.password-verify.php

- PHP Manual – MySQLi prepared statements: https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php
- PHP Manual – mysqli_report: https://www.php.net/manual/en/function.mysqli-report.php
- PHP Manual – random_bytes and hash_equals: https://www.php.net/manual/en/function.random-bytes.php, https://www.php.net/manual/en/function.hash-equals.php
- PHP Manual – Validate filters (FILTER_VALIDATE_EMAIL): https://www.php.net/manual/en/filter.filters.validate.php
- OWASP – Password Storage Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP – Cross-Site Request Forgery Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- OWASP – SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- MariaDB Knowledge Base – CONSTRAINT: https://mariadb.com/kb/en/constraint/
- Project repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 (folder Practical-9)